Basic Calculator: A Complete Derivation Log

> **Source:** A recurring question in Coupang interview reports (1point3acres). Equivalent to the union of LeetCode 224 (parentheses) and 227 (multiplication/division). This document records the **derivation process itself** — including three failed invariants and the specific defect each one exposed.

---

1. Problem

Evaluate a string expression:

- Contains non-negative integers, `+`, `-`, `*`, `/`, `(`, `)`, and spaces
- Integer division truncates toward zero
- The expression is guaranteed well-formed (no validation — an assumption worth **stating out loud** in an interview)

Examples: `3+2*2` → 7; `(1+(4+5+2)-3)+(6+8)` → 23; `2*(5+5*2)/3+(6/2+8)` → 21

---

2. Where the Difficulty Lies

The difficulty is **not in the algorithmic idea**. Three standard families solve it — three-level recursive descent, single stack with a pending operator, and the shunting-yard algorithm — all O(n), with no hidden insight.

The difficulty is **bookkeeping**: three orthogonal concerns must hold simultaneously.

Dimension	Requirement	Typical failure
Precedence	`* /` binds tighter than `+ -`	Recursion swallows the entire right side
Associativity	Same-level operators are left-associative	`8-3-2` evaluates to 7
Nesting	Parentheses nest arbitrarily and act as operands	Parentheses handled separately from operands

The three constrain each other: fixing precedence often breaks associativity, and vice versa. The derivation below is exactly that struggle.

---

3. The Derivation: Three Failed Invariants

3.1 Version 1 — `dfs(i)` returns "the value of everything to the right of i"

The most natural first idea: the recursive function returns the value of the whole expression from the current position to the end.

Test `3+2*2`: read 3 → read `+` → hand `2*2` to recursion, get 4 → `3+4 = 7`. ✓

Test `2*3+4`: read 2 → read `*` → hand `3+4` to recursion, get 7 → `2*7 = 14`. ✗ (expected 10)

Test `8-3-2`: read 8 → read `-` → hand `3-2` to recursion, get 1 → `8-1 = 7`. ✗ (expected 3)

Diagnosis: This invariant makes **every operator equal-precedence and right-associative**. "Swallow everything to the right" treats all operators alike, so there is neither a notion of precedence nor any opportunity to settle left to right.

One failed invariant exposed two defects, which means they share a root cause: **the recursion does not know how much it should consume.**

3.2 Version 2 — fold `* /` immediately, recurse on `+ -`

Fix: treat the two precedence levels differently. On `* /`, read exactly one operand and fold it into the running value. On `+ -`, hand

the rest to recursion.

Measured:

```
ok  2*3+4                    -> 10
ok  2+3*4                    -> 14
ok  1+2+3                    -> 6
ok  2*3*4                    -> 24
ok  8/2/2                    -> 2
ok  3+2*(5+5)/2              -> 13
ok  1-(5-2)                  -> -2
ok  2*(5+5*2)/3+(6/2+8)      -> 21
ok  (2+6*3+5-(3*14/7+2)*5)+3 -> -12
XX  3-1-2                    -> 4    (want 0)
XX  8-3-2                    -> 7    (want 3)
```

Precedence is fixed and parentheses work, but associativity is still broken.

Diagnosis: `+` may swallow the rest (addition is associative: `a+(b+c) = (a+b)+c`). `-` may not (`a-(b-c) ≠ (a-b)-c`). This is the first key fork in the problem.

3.3 Version 3 — drop the operator stack, fold `+ -` immediately too

Fix: if recursion causes right-associativity, don't recurse — fold `+ -` against the stack top on the spot as well.

Measured:

```
ok  3-1-2                    -> 0
ok  8-3-2                    -> 3
ok  2*3+4                    -> 10
ok  8/2/2                    -> 2
ok  1-(5-2)                  -> -2
ok  1000/2/2/2/5             -> 25
XX  2+3*4                    -> 20   (want 14)
XX  1+2*3+4                  -> 13   (want 11)
XX  14-3/2                   -> 5    (want 13)
XX  3+2*(5+5)/2              -> 25   (want 13)
XX  2*(5+5*2)/3+(6/2+8)      -> 24   (want 21)
XX  (2+6*3+5-(3*14/7+2)*5)+3 -> 108  (want -12)
```

Associativity is fixed; precedence collapses. Push down here, it pops up there.

Diagnosis: In `2+3*4`, reading 3 settles `2+3 = 5` on the spot, so by the time `*` arrives, the 3 is already consumed. The right operand of a `+ -` is **not yet determined** — it might be `3`, or `3*4*5*6`. Therefore `+ -` cannot settle immediately.

---

4. Three Key Insights

After three failures the problem compresses to one sentence: **`* /` must settle immediately, `+ -` must be deferred — deferred until when?**

Insight 1: defer until the next `+ -` arrives

"In `1+2+3+4`, once the last number arrives, the earlier additions can be settled."

Pushed to its conclusion: whenever a new `+ -` appears (or the expression ends, or a `)` is reached), the right operand of the previous additive term is complete and can be settled.

Implementation: **push** the pending term onto a stack, and sum the stack at the end.

Insight 2: a minus sign is not an operator, it is a negation

This is the cheapest step in the whole derivation.

If the `-` in `a-b` is read as "push `-b`", then:

- **No operator stack is needed** — the stack holds only numbers
- **Order becomes irrelevant** — `8-3-2` pushes `[8, -3, -2]`, summing to 3 regardless of direction
- **No reversal, no pairing**

Verification: `8-3-2` → stack `[8, -3, -2]` → sum 3. ✓

Insight 3: a parenthesised group is just an operand

`(1+2)*3` shows parentheses can appear as the left operand; `3*(1+2)` shows they can appear on the right. So parentheses are not a separate syntactic layer — they occupy the same slot as a number: **"read one operand" has two cases, a number or a parenthesised expression**, and the latter's value comes from the same logic, recursively.

Recursion is therefore used *only* for parentheses; `* /` and `+ -` are both handled in one loop, dispatched by a single variable.

A supporting trick: get the operand first

The derivation kept drifting back toward an operator stack. The root cause was **ordering**: if you read an operator and then think "remember it for later," you necessarily need storage.

Reverse the order — **read the operand first, then look back at the operator that preceded it**. At that moment you hold both the operator and the operand, so you can act in one step with nothing stashed.

The loop shape becomes "read an operand → check `lastop` → act," rather than "read an operator → stash it → wait for an operand."

---

5. Final Invariants

Function `eval()`:

- **Precondition:** `p` points at the first character of a (sub)expression
- **Postcondition:** `p` points at the `)` that closes it, or at end-of-string
- **Returns:** the value of that (sub)expression

Loop invariants:

1. The stack holds **numbers only**, never operators
2. Each stack element is a **completed multiplicative term**, carrying its sign
3. `lastop` records "the operator preceding the next operand," initialised to `+`
4. At any moment, the value of the scanned prefix equals the sum of the stack

Invariant 4 is the core: it guarantees that summing at the end yields the answer.

Three dispatch rules:

lastop	Action	Reason
`+`	`push(number)`	New additive term; right operand undetermined, defer
`-`	`push(-number)`	Same, with the minus folded into the sign
`*`	`pop` top `x`, `push(x * number)`	Highest precedence, settle now
`/`	`pop` top `x`, `push(x / number)`	Same; note `x` is the dividend

6. Pseudocode

```
eval(s, p):
    st = empty stack of numbers
    lastop = '+'
    while p < len(s):
        skip spaces
        c = s[p]

        if c == ')':  break                 # let the caller consume it

        if c is digit or c == '(':
            if c == '(':
                p++                         # consume '('
                number = eval(s, p)         # recurse; p lands on ')'
                p++                         # consume ')'
            else:
                number = parseNumber(s, p)

            if   lastop == '+':  st.push(number)
            elif lastop == '-':  st.push(-number)
            elif lastop == '*':  x = st.pop(); st.push(x * number)
            else:                x = st.pop(); st.push(x / number)
            continue

        if c in "+-*/":
            lastop = c
            p++
            continue

    return sum(st)
```

7. C++ Implementation

```cpp
class Calculator {
public:
    int calculate(const string& s) {
        S = &s;
        p = 0;
        return (int)eval();
    }
```

```
    private:
        const string* S = nullptr;
        size_t p = 0;

        void skipSpaces() { while (p < S->size() && (*S)[p] == ' ') ++p; }

        long long parseNumber() {
            long long x = 0;
            while (p < S->size() && isdigit((unsigned char)(*S)[p])) {
                x = x * 10 + ((*S)[p] - '0');
                ++p;
            }
            return x;
        }

        // pre:  p points at the first char of a subexpression
        // post: p points at the closing ')' or at end-of-string
        long long eval() {
            vector<long long> st;
            char lastop = '+';
            while (p < S->size()) {
                skipSpaces();
                if (p >= S->size()) break;
                char c = (*S)[p];

                if (c == ')') break;

                if (isdigit((unsigned char)c) || c == '(') {
                    long long number;
                    if (c == '(') {
                        ++p;                // consume '('
                        number = eval();    // p now sits on the matching ')'
                        ++p;                // consume ')'
                    } else {
                        number = parseNumber();
                    }
                    if      (lastop == '+') st.push_back(number);
                    else if (lastop == '-') st.push_back(-number);
                    else if (lastop == '*') { long long x = st.back(); st.pop_back(); st.push_back(x * number); }
                    else                    { long long x = st.back(); st.pop_back(); st.push_back(x / number); }
                    continue;
                }

                if (c=='+'||c=='-'||c=='*'||c=='/') { lastop = c; ++p; continue; }
                ++p;
            }
            long long sum = 0;
            for (long long v : st) sum += v;
            return sum;
        }
    };
```

A note on position management — this was the longest sticking point in the derivation.

`p` is a **member variable**, not "start as an in-parameter, end as an out-parameter." An earlier version used `dfs(s, start, int& end)`: the caller supplies the start, the callee writes back the end, and the two must reconcile. But `end` carried inconsistent

meanings across branches — sometimes the last consumed position, sometimes the next unread position, with stray `+1` offsets — and three meanings for one variable made the derivation impossible to carry out on paper.

With a shared cursor, the concept of `end` disappears entirely: each function advances `p` as far as it read, and the caller simply continues. **Still a single pass**, but with half the state.

---

8. Complexity

- **Time:** O(n) — each character is visited a constant number of times
- **Space:** O(n) — stack depth plus recursion depth for nested parentheses

The number of stack elements equals the number of additive terms at the current level; recursion depth equals parenthesis nesting depth.

---

9. Verification

30 hand-written cases plus 6,000 randomised differential cases, all passing:

```
hand cases: 30 total, 0 failed
random cases: 6000 total, 0 failed
```

The **reference implementation** for differential testing is a recursive-descent parser (expr / term / factor), **structurally independent** of the single-stack implementation under test. That independence is what lets it catch transition errors rather than reproducing the same bug twice.

Hand-written cases cover:

- Single numbers, all-whitespace, leading/trailing whitespace
- Precedence: `3+2*2`, `2*3+4`, `1+2*3+4`
- Left associativity: `3-1-2`, `8-3-2`, `8/2/2`, `1000/2/2/2/5`
- Division truncation: `3/2`, `1/2`, `14-3/2`
- Parentheses as left and right operands: `(1+2)*3`, `3*(1+2)`, `(1+2)*(3+4)`
- Deep nesting: `((((5))))`, `100/((2+3)*5)`
- Long mixed expressions: `2*(5+5*2)/3+(6/2+8)`, `(2+6*3+5-(3*14/7+2)*5)+3`

The generator builds well-formed expressions and rewrites every divisor as `(b*b+1)` to guarantee it is non-zero.

---

10. Comparing the Three Families

Approach	State	Code size	Notes
Single stack + lastop (this document)	One number stack, one `lastop`, one cursor	Smallest	Recursion only for parentheses
Three-level recursive descent	Three functions, one local value each	Medium	expr/term/factor; independent invariants per level, easiest to explain
Shunting-yard (two stacks)	Number stack + operator stack + precedence table	Largest	Fully general (arbitrary precedence and associativity); poor value in an interview

All three share the same asymptotic complexity; the choice is driven by **state count**, not performance. Every point lost on this problem is lost in the details, so less state is safer.

Worth noting: with only two precedence levels, the shunting-yard rule "compare the stack top against the incoming operator" degenerates into the single test "is the incoming operator `* /`?" — no precedence table required. One step further (minus as negation) and the operator stack disappears too. **The degenerate form of a general algorithm, under the problem's actual constraints, is often the best answer.**

---

11. Retrospective

The workflow that worked:

1. State the algorithm →
2. Write pseudocode until it is **fully closed** (no placeholder phrases like "handle this carefully") →
3. Translate to code →
4. Hand-trace three steps against the invariants

The first two steps catch algorithm-level errors, and those are the expensive ones: a syntax-level bug surfaces the moment you compile and run, whereas an algorithm-level bug is invisible without differential testing — and there is no differential testing in an interview room.

Every failure in this problem sat at the "invariant definition" layer, not the coding layer:

Version	Invariant	What broke
v1	dfs returns everything to the right	Precedence and associativity, together
v2	`* /` immediate, `+ -` recursive	Associativity
v3	Everything settles immediately	Precedence
v4	`* /` immediate, `+ -` deferred onto a stack	✓

General lesson: when one function carries two responsibilities with different invariants — here "consume a run of `* /`" and "consume a run of `+ -`" — no amount of tuning avoids the whack-a-mole. The way out is to separate them: either into two layers of functions, or into a recursion invariant and a loop invariant. This solution takes the latter route.

---

12. Bugs Found During Translation

Once the pseudocode was closed, translating it produced exactly three defects — all mechanical, none algorithmic:

Bug	Symptom	Class
Missing `i++` in the digit-parsing loop	Infinite loop on `1+1`	Cursor not advanced
Missing `i++` in the operator branch	Infinite loop	Cursor not advanced
No branch for the space character	Segfault on `"  7  "`	Unhandled character class

The third is worth generalising: **every character class named in the problem statement needs an explicit landing spot in the code.** The statement listed digits, five operators, parentheses, and spaces. The first three had branches; spaces fell through to the operator branch, were stored as `lastop`, and the cursor never moved.

A two-minute self-check covering three things — does every branch advance the cursor, does every named character class have a home, are all members initialised — would have caught all three.